

Framework and MVC Architecture

Use familiar PHP concepts inside a shared application structure.

Learning Goals

- Explain what a framework provides compared with the manual PHP of Module 1.
- Name the roles of Route, Controller, Model, View, and Migration, and where each lives in Laravel.
- Trace a `GET /api/students` request through the route, controller, and model.

What Is a Framework?

A framework provides structure and reusable tools for an application.

- You write the behavior specific to your project.
- Laravel supplies routing, database tools, and request handling.
- Composer installs PHP dependencies; Artisan runs Laravel commands.

Manual vs Automatic Server Processing

- In Module 1, we used PHP to handle server-side tasks manually.
- In Module 2, Laravel handles many of these tasks for us.
- To use Laravel effectively, we need to understand its MVC architecture and supporting tools.

What is MVC Architecture

MVC organizes an application into three cooperating roles: the `Model` manages data, the `View` presents output, and the `Controller` handles requests and coordinates the work.

- In Laravel, a `model` represents a type of application data and interacts with its database records.
- An API usually returns JSON instead of an HTML `view`.
- Laravel routes direct requests to `controllers`, while migrations define the database structure.

MVC: Three Responsibilities, One Location Each

Part	Responsibility	Laravel location
Route	Match method and URL to a controller	<code>routes/api.php</code>
Controller	Handle a request and return a response	<code>app/Http/Controllers/</code>
Model	Work with data and database records	<code>app/Models/</code>
View	Render an HTML page (not used in this module: our API returns JSON)	<code>resources/views/</code>
Database	Define database	

One Request Through MVC

```
GET /api/students
```

```
-> Route -> Controller -> Student Model -> MySQL  
<- JSON response to the client
```

- The route sends the request to a controller. The controller asks the model for data and returns JSON.
- This module focuses on APIs: every answer is JSON.
- This is automatically processed by the Laravel controller.

Controller - The Magic Behind the Automation

The Laravel controller maps the user request to the Laravel method that process the request.

- For example, `GET /api/students`, the route matches the URL, but the application must also fetch student records and return them as JSON.
- Putting all this code in the route mixes URL matching, data access, and response handling, making each harder to change (Module 1).

A controller coordinates (and automates) this request: it asks the model for data and chooses the response.

- The model remains responsible for working with student records.
- In `routes/api.php`, define the route without the `/api` prefix.

In this example, Laravel adds that prefix automatically, so `GET /api/students` calls `StudentController::index()` :

```
use App\Http\Controllers\StudentController;  
Route::get('/students', [StudentController::class, 'index']);
```

- Notice that `/students` , not `/api/students` is used to map the request and the method.

```
public function index()  
{  
    $students = Student::all();  
    return response()->json($students);  
}
```

The model fetches the records, and the controller returns JSON.

What Laravel Does Not Decide

You still design the endpoints and database, and you write validation rules, permissions, and tests.

- Conventions reduce repeated code.
- Learning those conventions takes practice.
- Installed versions and dependencies affect setup.

Framework tools support correct code; they do not guarantee it.

Check Your Understanding

For `GET /api/students`, identify:

1. The file that matches the URL to a controller method.
2. The component that reads student records.
3. Where you would change the JSON that is returned.

Next: Locate these files yourself.